Buổi 8

Danh sách liên kết kép

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Cấu trúc danh sách liên kết kép

1.1 Vì sao cần danh sách liên kết kép?
1.2 Cấu trúc dữ liệu và các thao tác cơ bản
1.3 Duyệt xuôi, duyệt ngược và tìm kiếm

2. Thêm phần tử vào DSLK kép

2.1 Thêm vào đầu và vào cuối danh sách
2.2 Thêm vào sau một nút
2.3 Thêm vào trước một nút và thêm có thứ tự

3. Hủy phần tử khỏi DSLK kép

3.1 Hủy phần tử đầu và cuối danh sách
3.2 Hủy một nút bất kỳ và hủy theo khóa

4. Sắp xếp và so sánh

4.1 Sắp xếp trên DSLK kép
4.2 So sánh DSLK đơn, DSLK kép và danh sách đặc
4.3 Ưu nhược điểm của danh sách liên kết
4.4 Ứng dụng DSLK vòng: bài toán Josephus
4.5 Ví dụ tổng hợp: lịch sử duyệt web

5. Tổng kết và thực hành

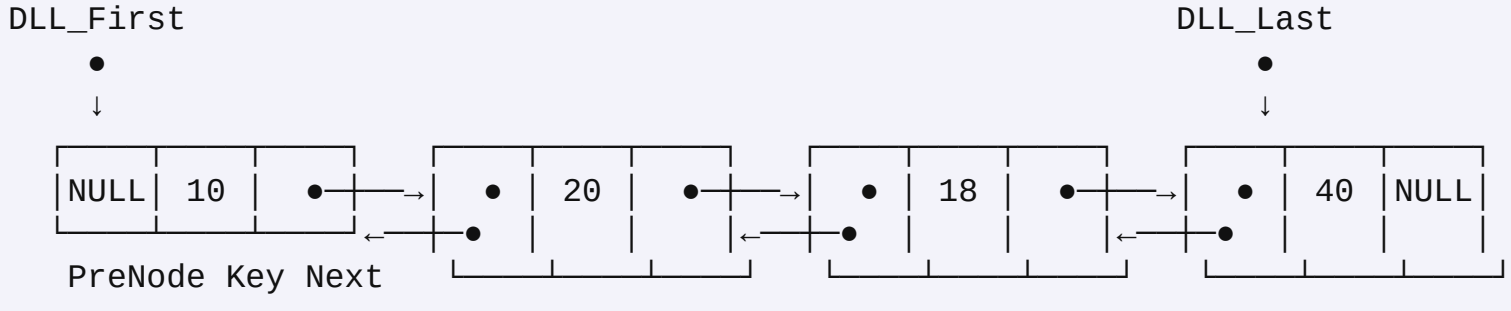
5.1 Bài thực hành buổi 8 – Chuyển chương trình buổi 7 sang DSLK kép
5.2 Câu hỏi và bài tập

1.1 Vì sao cần danh sách liên kết kép?

- **Hạn chế của DSLK đơn** (buổi 7):
  - Chỉ duyệt được **một chiều** (từ đầu đến cuối).
  - Muốn hủy một nút hoặc thêm vào **trước** một nút phải biết **nút đứng trước** ⇒ phải duyệt lại từ đầu O(N).
  - Hủy nút cuối O(N) ngay cả khi có con trỏ Last.

Định nghĩa

Danh sách liên kết kép (Doubly Linked List – DLL) là danh sách liên kết mà mỗi nút có **hai** liên kết: **NextNode** trỏ tới nút **kế sau** và **PreNode** trỏ tới nút **kế trước**. Danh sách được quản lý bởi hai con trỏ **DLL_First** (nút đầu) và **DLL_Last** (nút cuối).



- Nút đầu có PreNode = NULL; nút cuối có NextNode = NULL. Với mọi nút p ở giữa: **p->NextNode->PreNode == p** và **p->PreNode->NextNode == p** – tính chất bất biến phải giữ đúng sau mọi thao tác.

1.2 Cấu trúc dữ liệu và các thao tác cơ bản

```
typedef struct DLL_Node
{  T Key;
   DLL_Node *NextNode;     // liên kết tới nút sau
   DLL_Node *PreNode;      // liên kết tới nút trước
} DLL_OneNode;
typedef DLL_OneNode *DLL_NodePointer;
typedef struct
{  DLL_NodePointer DLL_First;
   DLL_NodePointer DLL_Last;
} DLL_Type;
```

- **Khởi tạo:**

```
void DLL_Initialize(DLL_Type &List)
{  List.DLL_First = List.DLL_Last = NULL;  }
```

- **Tạo nút mới:**

```
DLL_NodePointer DLL_Create_Node(T NewData)
{  DLL_NodePointer Pnode = new DLL_OneNode;
   if (Pnode != NULL)
   {  Pnode->NextNode = NULL;
      Pnode->PreNode  = NULL;
      Pnode->Key = NewData;
   }
   return (Pnode);
}
```

- **Bộ nhớ:** mỗi nút tốn thêm **một con trỏ** (8 byte trên máy 64-bit) so với DSLK đơn. Với T = int (4 byte), một nút DLL ≈ 24 byte (đã căn lề) so với ≈ 16 byte của SLL.

1.3 Duyệt xuôi, duyệt ngược và tìm kiếm

```
void DLL_Show_Forward(DLL_Type List)     // duyệt xuôi
{  DLL_NodePointer p = List.DLL_First;
   while (p != NULL)
   {  printf("%4d", p->Key);
      p = p->NextNode;
   }
}
void DLL_Show_Backward(DLL_Type List)    // duyệt ngược
{  DLL_NodePointer p = List.DLL_Last;
   while (p != NULL)
   {  printf("%4d", p->Key);
      p = p->PreNode;
   }
}
DLL_NodePointer DLL_Search(DLL_Type List, T X)
{  DLL_NodePointer p = List.DLL_First;
   while (p != NULL && p->Key != X)
      p = p->NextNode;
   return p;
}
```

Ví dụ 8.1 – Danh sách (5, 10, 20, 18, 40)

- Duyệt xuôi: **5 10 20 18 40**.
- Duyệt ngược: **40 18 20 10 5** – DSLK đơn muốn làm việc này phải dùng đệ quy hoặc đảo danh sách.

Ví dụ 8.2 – Tận dụng hai chiều

- Danh sách có thứ tự tăng, tìm X: nếu X gần giá trị lớn nhất, **tìm từ cuối** (DLL_Last, đi theo PreNode) sẽ nhanh hơn.
- Truy cập phần tử thứ i: nếu i > N/2 thì đi từ cuối ⇒ tối đa N/2 bước.
- Ứng dụng thực tế: nút **Back / Forward** của trình duyệt, **Undo / Redo**, danh sách bài hát (bài trước / bài sau).

2.1 Thêm vào đầu và vào cuối danh sách

```
B1: NewNode = DLL_Create_Node(NewData)
B2: IF (NewNode = NULL) Thực hiện Bkt
B3: IF (DLL_First = NULL)        // danh sách rỗng
      DLL_First = DLL_Last = NewNode
      Thực hiện Bkt
B4: NewNode->NextNode = DLL_First
B5: DLL_First->PreNode = NewNode
B6: DLL_First = NewNode
Bkt: Kết thúc
```

```
DLL_NodePointer DLL_Add_First(DLL_Type &List, T NewData)
{  DLL_NodePointer NewNode = DLL_Create_Node(NewData);
   if (NewNode == NULL) return (NULL);
   if (List.DLL_First == NULL)
      List.DLL_First = List.DLL_Last = NewNode;
   else
   {  NewNode->NextNode = List.DLL_First;
      List.DLL_First->PreNode = NewNode;
      List.DLL_First = NewNode;
   }
   return (NewNode);
}
```

- **Thêm vào cuối** – đối xứng, **O(1)** nhờ DLL_Last:

```
DLL_NodePointer DLL_Add_Last(DLL_Type &List, T NewData)
{  DLL_NodePointer NewNode = DLL_Create_Node(NewData);
   if (NewNode == NULL) return (NULL);
   if (List.DLL_Last == NULL)
      List.DLL_First = List.DLL_Last = NewNode;
   else
   {  List.DLL_Last->NextNode = NewNode;
      NewNode->PreNode = List.DLL_Last;
      List.DLL_Last = NewNode;
   }
   return (NewNode);
}
```

Ví dụ 8.3 – Thêm 5 vào đầu (10, 20, 18, 40)

B4: 5.Next → 10; B5: 10.Pre → 5; B6: DLL_First → 5 ⇒ **(5, 10, 20, 18, 40)**; DLL_Last vẫn là 40.

- So với SLL: thêm **1 phép gán** (cập nhật PreNode).

2.2 Thêm vào sau một nút

- Thêm NewData vào **sau** nút InsNode (InsNode không phải nút cuối): phải cập nhật **4 liên kết**.

```
B1: NewNode = DLL_Create_Node(NewData)
B2: IF (NewNode = NULL) Thực hiện Bkt
B3: NewNode->NextNode = InsNode->NextNode   // (1)
B4: NewNode->PreNode  = InsNode             // (2)
B5: InsNode->NextNode->PreNode = NewNode    // (3)
B6: InsNode->NextNode = NewNode             // (4)
Bkt: Kết thúc
```

```
DLL_NodePointer DLL_Add_After(DLL_Type &List, T NewData,
                              DLL_NodePointer InsNode)
{  if (InsNode == List.DLL_Last)
      return DLL_Add_Last(List, NewData);
   DLL_NodePointer NewNode = DLL_Create_Node(NewData);
   if (NewNode == NULL) return (NULL);
   NewNode->NextNode = InsNode->NextNode;
   NewNode->PreNode  = InsNode;
   InsNode->NextNode->PreNode = NewNode;
   InsNode->NextNode = NewNode;
   return (NewNode);
}
```

Ví dụ 8.4 – Thêm 25 vào sau nút 20 trong (5, 10, 20, 18, 40)

```
Trước:   … [10] ⇄ [20] ⇄ [18] ⇄ [40]
                   InsNode
(1),(2): NewNode [25]: Next → 18, Pre → 20
         … [10] ⇄ [20] ⇄ [18] ⇄ [40]
                     ↖ [25] ↗
(3):     18.Pre → 25
(4):     20.Next → 25
Sau:     … [10] ⇄ [20] ⇄ [25] ⇄ [18] ⇄ [40]
```

Thứ tự gán

Bước (4) phải làm **sau** bước (3): nếu gán InsNode->NextNode = NewNode trước, thì `InsNode->NextNode->PreNode` ở bước (3) sẽ trỏ vào chính NewNode ⇒ nút 18 **không** được cập nhật PreNode.

2.3 Thêm vào trước một nút và thêm có thứ tự

- **Thêm vào trước nút InsNode** – điều DSLK đơn làm rất khó, nay chỉ O(1):

```
DLL_NodePointer DLL_Add_Before(DLL_Type &List, T NewData,
                               DLL_NodePointer InsNode)
{  if (InsNode == List.DLL_First)
      return DLL_Add_First(List, NewData);
   return DLL_Add_After(List, NewData, InsNode->PreNode);
}
```

- Ý tưởng: "thêm trước InsNode" = "thêm sau nút đứng trước InsNode" – có ngay nhờ **PreNode**.

Ví dụ 8.5

Danh sách (5, 10, 20, 25, 18, 40): thêm 1 vào trước nút 5 (nút đầu) ⇒ gọi Add_First; thêm 99 vào sau nút 40 (nút cuối) ⇒ gọi Add_Last ⇒ **(1, 5, 10, 20, 25, 18, 40, 99)**; duyệt ngược: 99 40 18 25 20 10 5 1 ✓.

- **Thêm vào danh sách có thứ tự tăng:**

```
void DLL_Add_Order(DLL_Type &List, T X)
{  DLL_NodePointer p = List.DLL_First;
   while (p != NULL && p->Key <= X)
      p = p->NextNode;
   if (p == NULL)
      DLL_Add_Last(List, X);        // X lớn nhất
   else
      DLL_Add_Before(List, X, p);   // chèn trước nút đầu
tiên > X
}
```

- So với SLL_Add_Order (buổi 7): không cần "nhìn trước" `CurNode->NextNode->Key`, code tự nhiên hơn.
- **Cải tiến:** nếu X ≥ DLL_Last->Key ⇒ thêm cuối ngay O(1) (thường gặp khi dữ liệu đến gần theo thứ tự).

3.1 Hủy phần tử đầu và cuối danh sách

```
int DLL_Delete_First(DLL_Type &List)
{  DLL_NodePointer TempNode = List.DLL_First;
   if (TempNode == NULL) return (0);
   List.DLL_First = TempNode->NextNode;
   if (List.DLL_First == NULL)       // danh sách thành rỗng
      List.DLL_Last = NULL;
   else
      List.DLL_First->PreNode = NULL;
   delete TempNode;
   return (1);
}
```

```
int DLL_Delete_Last(DLL_Type &List)
{  DLL_NodePointer TempNode = List.DLL_Last;
   if (TempNode == NULL) return (0);
   List.DLL_Last = TempNode->PreNode;
   if (List.DLL_Last == NULL)        // danh sách thành rỗng
      List.DLL_First = NULL;
   else
      List.DLL_Last->NextNode = NULL;
   delete TempNode;
   return (1);
}
```

Ví dụ 8.6 – Hủy cuối danh sách (5, 10, 20, 18, 40)

Bước	Thao tác	Trạng thái
1	TempNode = DLL_Last (nút 40)	
2	DLL_Last = TempNode->PreNode (nút 18)	DLL_Last → 18
3	DLL_Last->NextNode = NULL	18 trở thành nút cuối
4	delete TempNode	(5, 10, 20, 18)

- Chỉ **O(1)** – DSLK đơn cần O(N) để tìm nút đứng trước nút cuối.
- **Trường hợp biên:** danh sách chỉ có **1 nút** ⇒ sau khi hủy DLL_First = DLL_Last = **NULL**. Quên xử lý ⇒ DLL_First vẫn trỏ vào nút đã giải phóng (con trỏ treo).

3.2 Hủy một nút bất kỳ và hủy theo khóa

- **Hủy nút DelNode** (biết địa chỉ) – không cần nút trước vì đã có **PreNode**:

```
int DLL_Delete_Node(DLL_Type &List, DLL_NodePointer DelNode)
{  if (DelNode == NULL) return 0;
   if (DelNode == List.DLL_First)
      return DLL_Delete_First(List);
   if (DelNode == List.DLL_Last)
      return DLL_Delete_Last(List);
   DelNode->PreNode->NextNode = DelNode->NextNode;  // (1)
   DelNode->NextNode->PreNode = DelNode->PreNode;   // (2)
   delete DelNode;
   return 1;
}
```

- **Hủy theo khóa X:** `DLL_Delete_Node(List, DLL_Search(List, X));`

Ví dụ 8.7 – Hủy nút 25 trong (1, 5, 10, 20, 25, 18, 40, 99)

```
Trước:  [20] ⇄ [25] ⇄ [18]
(1):    20.Next → 18       [20] → [18],  [25] vẫn trỏ tới 2 bên
(2):    18.Pre  → 20       [20] ⇄ [18]
delete: giải phóng nút 25
```

- Tiếp tục hủy 1 (nút đầu) và 99 (nút cuối) ⇒ **(5, 10, 20, 18, 40)**; duyệt ngược 40 18 20 10 5 ✓.

- **Hủy toàn bộ danh sách:**

```
void DLL_Delete(DLL_Type &List)
{  while (DLL_Delete_First(List));  }
```

4.1 Sắp xếp trên DSLK kép

- **Cách 1 – hoán vị dữ liệu** (giống SLL), ví dụ Selection Sort:

```
void DLL_Selection_Sort(DLL_Type List)
{  for (DLL_NodePointer Ki = List.DLL_First;
        Ki != List.DLL_Last; Ki = Ki->NextNode)
   {  DLL_NodePointer Min = Ki;
      for (DLL_NodePointer Kj = Ki->NextNode;
           Kj != NULL; Kj = Kj->NextNode)
         if (Kj->Key < Min->Key) Min = Kj;
      T t = Ki->Key; Ki->Key = Min->Key; Min->Key = t;
   }
}
```

- (5, 10, 20, 18, 40) ⇒ **(5, 10, 18, 20, 40)**.

- **Cách 2 – Insertion Sort "thật"** trên DLL: dịch con trỏ lùi theo **PreNode** giống hệt Insertion Sort trên mảng (lùi `Pos--`) – điều SLL không làm được.

```
void DLL_Insertion_Sort(DLL_Type List)
{  for (DLL_NodePointer i = List.DLL_First ? List.DLL_First-
>NextNode : NULL;
        i != NULL; i = i->NextNode)
   {  T X = i->Key;
      DLL_NodePointer j = i->PreNode;
      while (j != NULL && j->Key > X)
      {  j->NextNode->Key = j->Key;   // "dời" sang phải
         j = j->PreNode;
      }
      if (j == NULL) List.DLL_First->Key = X;
      else j->NextNode->Key = X;
   }
}
```

- Có thể cài **QuickSort** trên DLL với hai con trỏ i (đi xuôi) và j (đi ngược) – tương tự hai chỉ số i, j trên mảng.

4.2 So sánh DSLK đơn, DSLK kép và danh sách đặc

Thao tác	Danh sách đặc	DSLK đơn (First, Last)	DSLK kép (First, Last)
Truy cập phần tử thứ i	O(1)	O(i)	O(min(i, N − i))
Thêm đầu / hủy đầu	O(N)	O(1)	O(1)
Thêm cuối	O(1)	O(1)	O(1)
Hủy cuối	O(1)	**O(N)**	**O(1)**
Thêm sau nút đã biết	O(N)	O(1)	O(1)
Thêm trước / hủy nút đã biết	O(N)	**O(N)** (tìm nút trước)	**O(1)**
Duyệt ngược	O(N)	Khó (đệ quy / đảo)	O(N) tự nhiên
Bộ nhớ phụ mỗi phần tử	0	1 con trỏ	2 con trỏ
Độ phức tạp cài đặt	Đơn giản	Trung bình	Phức tạp hơn (4 liên kết)

Ví dụ 8.8 – Chọn cấu trúc

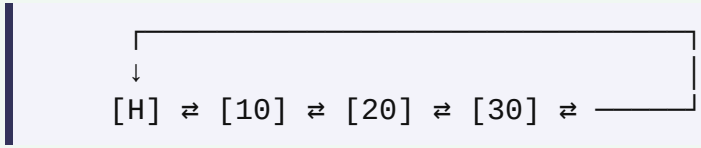
- Danh sách phát nhạc có "bài trước / bài sau", xóa bài đang phát ⇒ **DSLK kép**.
- Lịch sử tin nhắn chỉ thêm cuối, đọc từ đầu ⇒ **DSLK đơn** có Last (tiết kiệm bộ nhớ).
- Danh mục 50 tỉnh thành, tra cứu theo chỉ số ⇒ **danh sách đặc**.
- Trình soạn thảo văn bản: mỗi dòng là một nút, con trỏ dòng hiện tại di chuyển lên/xuống, chèn/xóa dòng tại chỗ ⇒ **DSLK kép**.

4.3 Ưu nhược điểm của danh sách liên kết

- **Ưu điểm** (so với danh sách đặc):
  - Kích thước **linh hoạt**, không cần biết trước MaxLen, không lãng phí vùng dự trữ.
  - Thêm/hủy **O(1)** khi đã biết vị trí – chỉ đổi liên kết, không dời dữ liệu.
  - Dễ tách, nhập, sắp xếp bằng đổi liên kết (không sao chép dữ liệu lớn).
- **Nhược điểm:**
  - Tốn bộ nhớ cho con trỏ (với T nhỏ như int, con trỏ còn lớn hơn dữ liệu).
  - **Không truy cập ngẫu nhiên** ⇒ không tìm nhị phân, truy cập phần tử thứ i chậm.
  - Các nút rải rác trong bộ nhớ ⇒ kém tận dụng bộ nhớ đệm CPU (cache) ⇒ thực tế duyệt chậm hơn mảng.
  - Cài đặt phức tạp, dễ lỗi con trỏ, rò rỉ bộ nhớ.

- **Biến thể khác:**
  - **DSLK vòng (circular):** nút cuối trỏ về nút đầu ⇒ duyệt vòng (lập lịch xoay vòng, trò chơi Josephus).
  - **DSLK có nút đầu giả (header/sentinel):** luôn có một nút rỗng ở đầu ⇒ không cần xử lý riêng trường hợp danh sách rỗng hay thêm/hủy đầu.

Ví dụ 8.9 – DSLK kép vòng có nút giả H



- Rỗng ⇔ H->NextNode == H. Thêm sau bất kỳ nút nào (kể cả H) dùng **cùng một đoạn mã** 4 lệnh, không cần `if` cho đầu/cuối.

4.4 Ứng dụng DSLK vòng: bài toán Josephus

- **Bài toán:** N người đứng vòng tròn, đánh số 1..N; đếm từ người 1, cứ đến người thứ k thì loại người đó, đếm tiếp từ người kế. Tìm người còn lại.

```
int Josephus(int N, int k)
{  SLL_Type First = NULL, Last = NULL;
   for (int i = 1; i <= N; i++)        // tạo DSLK
   {  SLL_Type p = SLL_Create_Node(i);
      if (First == NULL) First = p;
      else Last->NextNode = p;
      Last = p;
   }
   Last->NextNode = First;             // khép vòng
   SLL_Type Pre = Last;
   while (Pre->NextNode != Pre)        // còn > 1 người
   {  for (int c = 1; c < k; c++) Pre = Pre->NextNode;
      SLL_Type Del = Pre->NextNode;    // người thứ k
      Pre->NextNode = Del->NextNode;
      delete Del;
   }
   int KetQua = Pre->Key;
   delete Pre;
   return KetQua;
}
```

- **Mô phỏng N = 7, k = 3:**

Lượt	Vòng hiện tại	Bị loại
1	1 2 **3** 4 5 6 7	3
2	4 5 **6** 7 1 2	6
3	7 1 **2** 4 5	2
4	4 5 **7** 1	7
5	1 4 **5**	5
6	1 4 **1**	1
Kết quả	**4**	

- Thứ tự loại: 3, 6, 2, 7, 5, 1 ⇒ người còn lại là **4**.
- **Vì sao dùng DSLK vòng?** Loại một người là **O(1)** (đổi một liên kết), vòng tròn được mô phỏng tự nhiên; dùng mảng phải dời phần tử hoặc đánh dấu và bỏ qua nhiều lần.
- Độ phức tạp O(N·k).

4.5 Ví dụ tổng hợp: lịch sử duyệt web (Back / Forward)

- **Bài toán:** trình duyệt lưu các trang đã xem; nút **Back** về trang trước, **Forward** sang trang sau; khi đang ở giữa lịch sử mà mở trang mới thì **xóa** toàn bộ các trang phía sau.

```
typedef struct Page {
   char Url[100];
   Page *NextNode, *PreNode;
} Page;
Page *Cur = NULL;                    // trang hiện tại
void MoTrang(const char *url)
{  // xóa mọi trang phía sau Cur
   Page *p = Cur ? Cur->NextNode : NULL;
   while (p != NULL)
   {  Page *q = p->NextNode;  delete p;  p = q;  }
   Page *n = new Page;
   strcpy(n->Url, url);
   n->NextNode = NULL;  n->PreNode = Cur;
   if (Cur) Cur->NextNode = n;
   Cur = n;
}
void Back()    { if (Cur && Cur->PreNode)  Cur = Cur-
>PreNode;  }
void Forward() { if (Cur && Cur->NextNode) Cur = Cur-
>NextNode; }
```

Ví dụ 8.11 – Chuỗi thao tác

Thao tác	Lịch sử	Trang hiện tại
Mở A, B, C	A ⇄ B ⇄ C	C
Back	A ⇄ B ⇄ C	B
Back	A ⇄ B ⇄ C	A
Forward	A ⇄ B ⇄ C	B
Mở D	A ⇄ B ⇄ D (C bị xóa)	D
Forward	A ⇄ B ⇄ D	D (không có trang sau)

- Back / Forward là **O(1)** nhờ hai liên kết; với DSLK đơn, Back phải duyệt lại từ đầu.
- Cùng ý tưởng: **Undo / Redo** trong trình soạn thảo.

5.1 Bài thực hành buổi 8 – Chuyển chương trình buổi 7 sang DSLK kép

Yêu cầu

```
typedef struct SV_DNode {
   SinhVien Data;
   SV_DNode *NextNode, *PreNode;
} SV_DOneNode;
typedef struct { SV_DOneNode *First, *Last; } SV_DList;
```

- 1) Viết lại các chức năng buổi 7 (nhập, xuất, tìm, thêm có thứ tự, xóa theo mã, xóa SV có DTB < 5, sắp xếp, hủy) trên DSLK kép.
- 2) Thêm chức năng: **xuất ngược** danh sách; thêm SV vào **trước** SV có mã cho trước; xóa SV **cuối** danh sách.
- 3) Viết hàm `KiemTra(SV_DList L)` duyệt và xác nhận tính bất biến: với mọi nút p ≠ Last: `p->NextNode->PreNode == p`; First->PreNode == NULL; Last->NextNode == NULL. Gọi hàm này sau **mỗi** thao tác trong chế độ gỡ lỗi.
- 4) So sánh số dòng code và số lần duyệt danh sách giữa hai phiên bản (SLL – DLL) cho thao tác "xóa SV cuối" và "thêm trước SV có mã X".

Gợi ý gỡ lỗi

In danh sách theo **cả hai chiều** sau mỗi thao tác: nếu duyệt xuôi đúng mà duyệt ngược sai (hoặc ngược lại) ⇒ đã quên cập nhật PreNode (hoặc NextNode) ở đâu đó.

5.2 Câu hỏi và bài tập

Câu hỏi

- 1) Trình bày cấu trúc nút của DSLK kép. Tính chất bất biến nào phải luôn đúng?
- 2) Khi thêm một nút vào giữa DSLK kép phải cập nhật mấy liên kết? Thứ tự cập nhật có quan trọng không? Vì sao?
- 3) Những thao tác nào trên DSLK kép nhanh hơn DSLK đơn? Cái giá phải trả là gì?

Bài tập (có đáp số)

- 1) Cho DLL = (8, 3, 15). Thực hiện: thêm 6 vào đầu; thêm 10 vào sau nút 3; hủy nút cuối; thêm 1 vào trước nút 8. Ghi kết quả duyệt xuôi và ngược. → Xuôi **(6, 1, 8, 3, 10)**; ngược **(10, 3, 8, 1, 6)**.
- 2) Viết hàm **đảo ngược** DSLK kép tại chỗ (gợi ý: với mỗi nút hoán đổi NextNode và PreNode, cuối cùng hoán đổi First và Last).
- 3) Viết hàm kiểm tra DSLK kép các số nguyên có **đối xứng** không (ví dụ 1 2 3 2 1) dùng hai con trỏ đi từ hai đầu.
- 4) Cài đặt trình soạn thảo dòng đơn giản: mỗi dòng là một nút; lệnh `i` (chèn dòng sau dòng hiện tại), `d` (xóa dòng hiện tại), `u`/`n` (lên/xuống), `p` (in toàn văn bản).